

Max Flow 1

Max-Flow

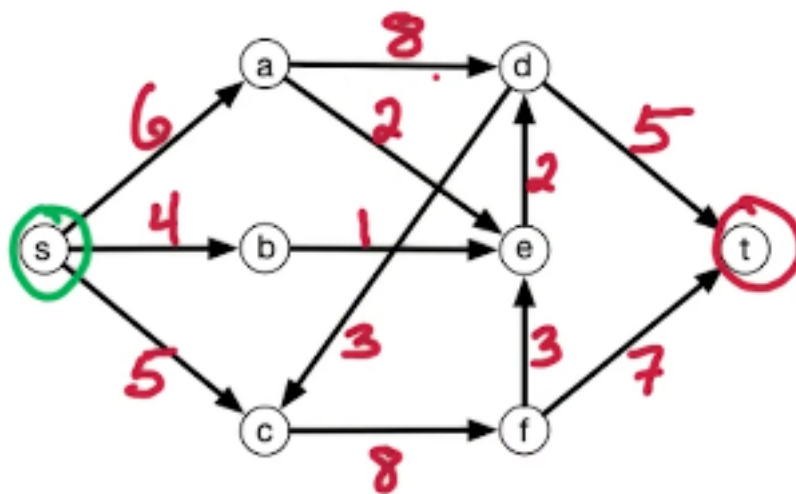
1. Ford-Fulkerson Algorithm
2. Max-Flow = Min-Cut
3. Image Segmentation
4. Max-Flow with Demands

Ford-Fulkerson Algorithm Lecture Outline

1. Algorithms
 - Ford-Fulkerson
 - Edmonds-Karp
2. Max-flow = min-cut theorem
3. Applications
 - Image segmentation: Want to separate image into foreground and background components

Problem Setup

1. Setting: Sending supply from vertex s to t
 - Maximize amount sent without exceeding edge capacities



Problem Setup

Problem Formulation

1. Flow network: Directed graph $G=(V,E)$, designated s,t in V where s and t are source and sink
 - For each edge in E , capacity $c_e < \infty$
2. Goal: Maximize flow from s to t where f_e is a flow along e

Max-Flow Problem

1. Input:
 - Flow network: directed $G=(V,E)$ with s,t in V and capacities $c_e > 0$ for e in E
2. Goal:
 - Find flows f_e for e in E where:

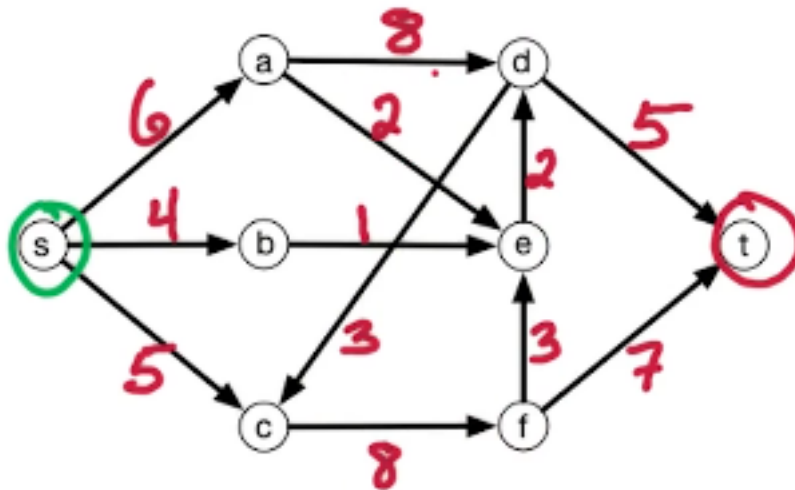
- Capacity constraint: for all e in E , $0 \leq f_e \leq c_e$
- Conservation of flow: for all v in $V - \{s \cup t\}$, flow-in to v = flow-out of v
 - * flow-in = $\sum(f_{wv})$ where edge wv enters node V
 - * flow-out = $\sum(f_{vz})$ where edge vz exits node V

Max-Flow Goal

1. Goal: Find a valid flow of maximum size
 - Size(of) = total flow sent
 - flow-out of s or flow-in to t

Quiz: Max-Flow Example

1. Find a flow of maximum size satisfying constraints
 - $sa = 6$
 - $sb = 1$
 - $sc = 5$
 - $ad = 6$
 - $ae = 0$
 - $be = 1$
 - $cf = 7$
 - $dc = 2$
 - $dt = 5$
 - $ed = 1$
 - $fe = 0$
 - $ft = 7$



Quiz 1

Cycles are OK

1. Cycle from C to F to E to D and back to C
 - Max-flow will ignore cycles
 - Suppose one unit of flow entered d
 - Could traverse the cycle, but this will never increase the flow
 - Instead, just pass the flow along
 - Problem is well-defined regardless of if there are cycles or not

Anti-parallel Edges

1. Notice that there are edges from A to B and B to A
 - Want to remove anti-parallel edges
 - Can simplify by introducing an intermediate vertex



Anti-parallel Edges

Toy Example

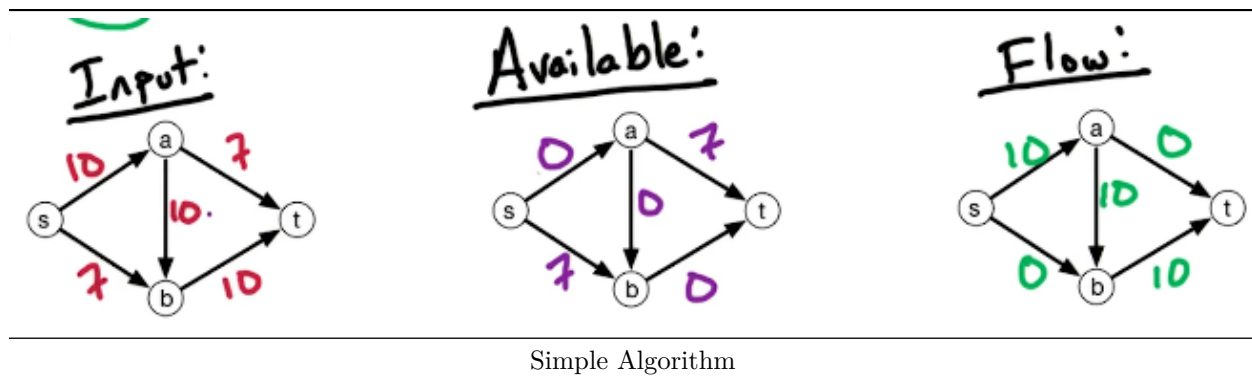
1. Use this network to devise algorithm



Toy Example (L) and Max-flow (R)

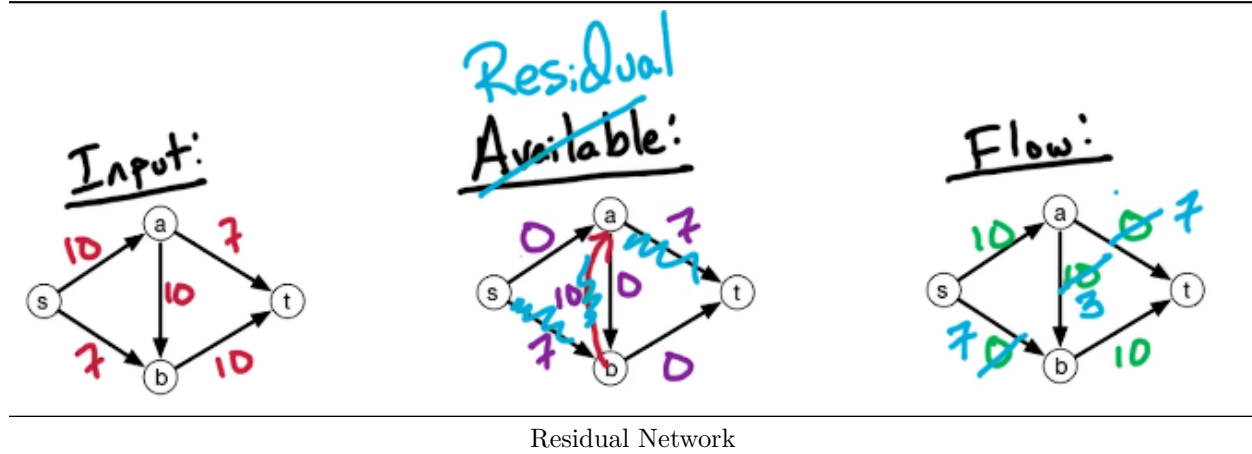
Simple Algorithm

1. Algorithm idea:
 - Start with $f_e = 0$ for all e in E
 - Find st-path with available capacity
 - Let $c(P) = \min(c_e - f_e)$ for all e in P
 - Max-flow along a path
 - Augment f by $c(P)$ along P
 - Repeat until no st-path
2. This algorithm will not produce the maximum flow in general



Backward Edges

- Need to add a backward edge from B to A of capacity 10
 - This corresponds to not using the path from A to B
 - Called the residual network
 - Use residual network instead of network of available capacities



Residual Network

- Definition of residual network:
 - For flow network $G=(V,E)$ with c_e for and flow f_e for e in E
 - $G^f = (V, E^f)$
 - If vw in E and $f_{vw} < c_{vw}$ then add vw to G^f with capacity $c_{vw} - f_{vw}$
 - If vw in E and $f_{vw} > 0$ then add wv to G^f with capacity f_{vw}

Ford-Fulkerson Algorithm

- Set $f_e = 0$ for all e in E
- Build the residual network G^f for current flow f
- Check for a st -path P in G^f
 - If no such path then output(f)
- Given P , let $c(P) = \min$ capacity along P in G^f
- Augment f by $c(P)$ units along P
 - For every forward edge, we increase the flow by this amount
 - For every backward edge, we decrease the flow by this amount
- Repeat until no such st path

Running Time

1. Correctness: Follows from max-flow = min-cut theorem
2. Running time: Assume all capacities are integers
 - Edmonds-Karp eliminates this assumption
 - When we augment the flow, augment it by an integer amount
 - Then, flow increases by ≥ 1 unit per round
 - Let C = size of max flow
 - Then $\leq C$ rounds since flow increases by ≥ 1 unit each round

Time per Round

1. Updating residual network takes $O(n)$ time
2. Checking for a st-path is $O(n+m) = O(m)$
3. Time per round
 - $O(m)$ time per round
 - Runtime: $O(mC)$ where C is size of max-flow

Discussion

1. Ford-Fulkerson: $O(mC)$ time
 - Where C = size of max flow assuming integer capacities
 - Pseudo-polynomial, like knapsack
2. Edmonds-Karp: $O(m^2n)$ time
 - Take shortest path from s to t where shortest means minimum number of edges
 - Just run BFS
 - From 1972
3. Orlin: $O(mn)$ time
 - From 2013